inc-mb-fidelity-inc___fidelity-flat-f-k4-inc

An AgentMPI run analysis

Abstract

This is the manager-summarises-everything cell of the enforced, incompressible reduction, and it is the run in which whole-rank erasure can be watched happening. Eight ranks send twelve high-entropy items each to rank 0, which folds them one at a time under a 450-token contract. The reported retention is 0.385, identical to its three siblings. What only this cell shows is the moment of failure: the root's accumulator reaches the budget after folding in rank 3, and the four merges that follow — `merge:d4` through `merge:d7`, the applications whose entire purpose is to bring in ranks 4, 5, 6 and 7 — each return the byte-identical blob `9f300f3a45bf`, 437 tokens, the same 37 identifiers. Four consecutive applications of the reduction operator were the identity function. They cost 89 s of wall clock and 2,888 prompt tokens, each satisfied its contract, each reported success, and together they produced nothing. Note also that `flat` is not a one-shot p-way fold: `algorithms.py` applies the binary operator $p-1$ times in sequence at the root, so this run's `fold_depth` is 7 with `rounds: 1`, and it is a depth-7 cell. This run also carries the campaign's only contract violation, and the retry is instructive in its own right: attempt 1 of `merge:d3` returned 472 tokens with 40 identifiers, was rejected for overrunning 450, and attempt 2 returned 437 with 37 — enforcement bought budget compliance by deleting three more items.

1 What this run is

property	value
run	`inc-mb-fidelity-inc__fidelity-flat-f-k4-inc`
experiment	-
campaign label	-
declared world size	8
ranks appearing in the log	8
executors	`broker` × 8, `worker` × 31
events	104
wall time	251.64 s
job outcome	completed

2 Headline measurements

metric	value	meaning
achieved parallelism	0.83×	total busy time over wall time
parallel efficiency	10.4%	achieved parallelism per rank
idle fraction	89.6%	rank-seconds paid for and unused
peak ranks busy	1	of 8 in the world
serial fraction of active time	100.0%	time with exactly one rank busy
load imbalance	8.00×	slowest rank's busy time over the mean
coordination share	12.5%	rank-seconds blocked in collectives, of those available
coordination in flight	100.0%	wall clock with any rank inside a collective
rank reattachments	31	fresh agent processes that took over a rank role
agent calls	7	invocations that reached a model
agent latency p50 / p95	25.01 / 74.93 s	per invocation
messages	7	7 eager, 0 rendezvous
tokens sent	1,106	0 deferred under rendezvous
tokens in / out	4,643 / 2,900	prompt and completion
cost	$0.0574	0.0198 $/kTok out

3 What the analysis notices

- [critical] At least one rank waited 7 s for an agent to claim its task before the broker gave up. That is a pool-sizing failure outside the protocol, not a slow run.
- [warning] Concurrency never exceeded one rank despite a world size of 8: this ran serially.
- [warning] The cost model's α and β here are a median fallback, not a fit: the slope was positive but the intercept was negative ($\alpha = -10.63$ s), so the fitted line passes below the origin. That is physically impossible and statistically unremarkable on a narrow token range, and the guard rejects it rather than publish a negative fixed cost, on a fit with $R^2 = 0.062$. Any latency prediction from this run's calibration is a median, not a model.
- [note] Load imbalance is 8.00×: the slowest rank was busy far longer than the mean.
- [note] 1 contract violation(s): an agent returned something the declared schema rejected.
- [note] 1 agent call(s) needed more than one attempt.
- [note] 1 trouble event(s) occurred but the run still completed: the harness absorbed them.
- [ok] 31 rank reattachment(s) occurred, up to incarnation 13: agent processes were replaced mid-run while the rank roles, their mailboxes, and their context accounts persisted.
- [ok] All 1 checkable collective(s) sent exactly the number of messages their closed-form cost expression predicts.
- [ok] All 1 checkable collective(s) also matched the predicted round count.

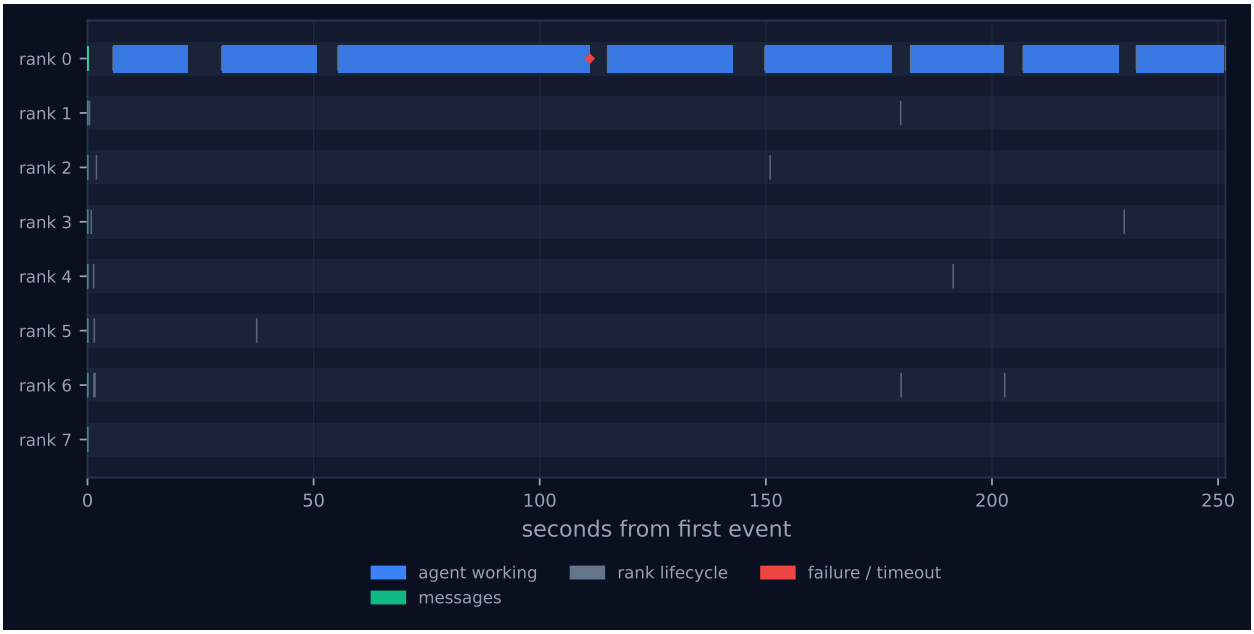

Figure 1: Execution timeline, one lane per rank. Bars are intervals when a rank held a task; ticks are messages; diamonds are window operations, lifecycle transitions, failures, and recovery actions. Drawn in the trace viewer's palette so the same shapes read the same way in both.

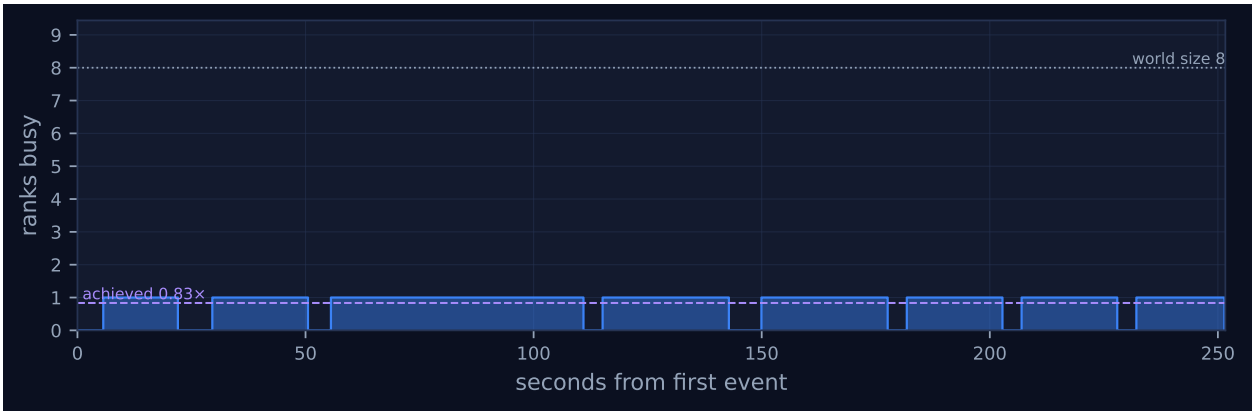

Figure 2: Ranks simultaneously busy over time. The dotted line is the declared world size and the dashed line the achieved parallelism; the gap between them is what the run paid for and did not use. Faint vertical lines mark collective invocations.

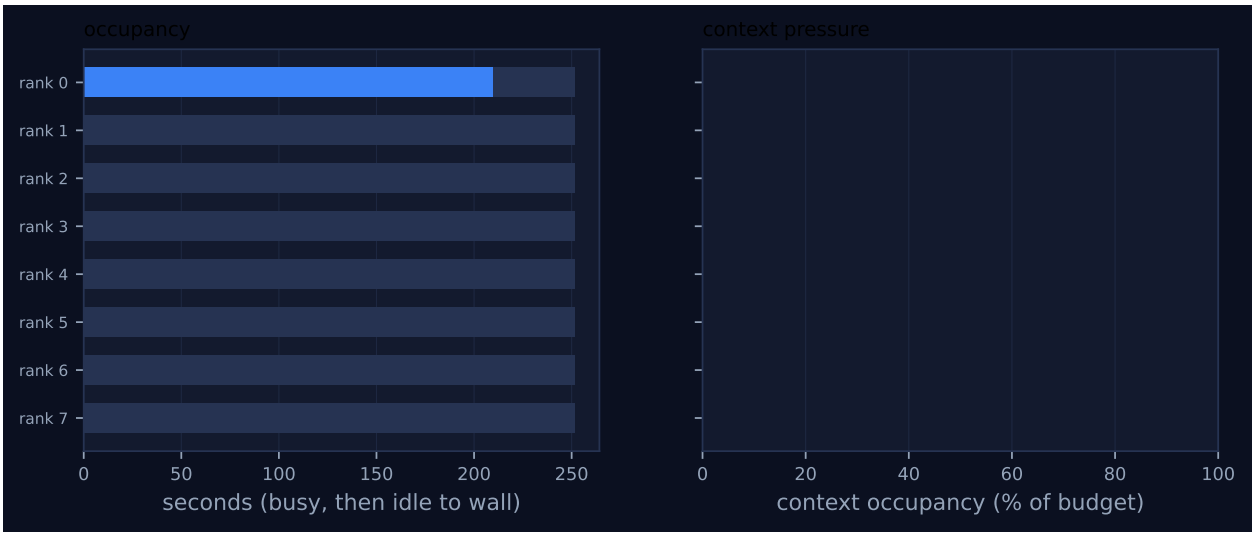

Figure 3: Per-rank occupancy and context pressure. Left: busy time, with the remainder to wall time in grey. Right: context occupancy against budget, amber above half and red above 80%, where admission pressure starts to reject artefacts.

4 Per-rank detail

rank	busy (s)	occ. (%)	tasks	calls	sent	recv	tok. sent	ctx (%)	state
0	209.91	57.6	8	7	0	7	0	0.0	finalized
1	0.00	0.0	0	0	1	0	158	0.0	finalized
2	0.00	0.0	0	0	1	0	158	0.0	finalized
3	0.00	0.0	0	0	1	0	158	0.0	finalized
4	0.00	0.0	0	0	1	0	158	0.0	finalized
5	0.00	0.0	0	0	1	0	158	0.0	finalized
6	0.00	0.0	0	0	1	0	158	0.0	finalized
7	0.00	0.0	0	0	1	0	158	0.0	finalized

Per-rank behaviour. Occupancy is busy time over the rank's own lifetime, so a rank that joined late is not penalised for the time before it existed.

5 Collectives, against the cost model

op	algorithm	label	p	seen	rounds	pred.	msgs	pred.	tokens	wall (s)	skew (s)	model
reduce	flat	–	8	8	1	1	7	7	1,106	251.63	251.63	✓

Messages are the count the fabric logged, attributed to each invocation through its internal tag, rather than the count the collective reported — the two are independently produced, and preferring the log is what makes this a check rather than a restatement. A dash in the model column means the comparison is not meaningful: either no closed form exists for that algorithm, or the invocation never completed.

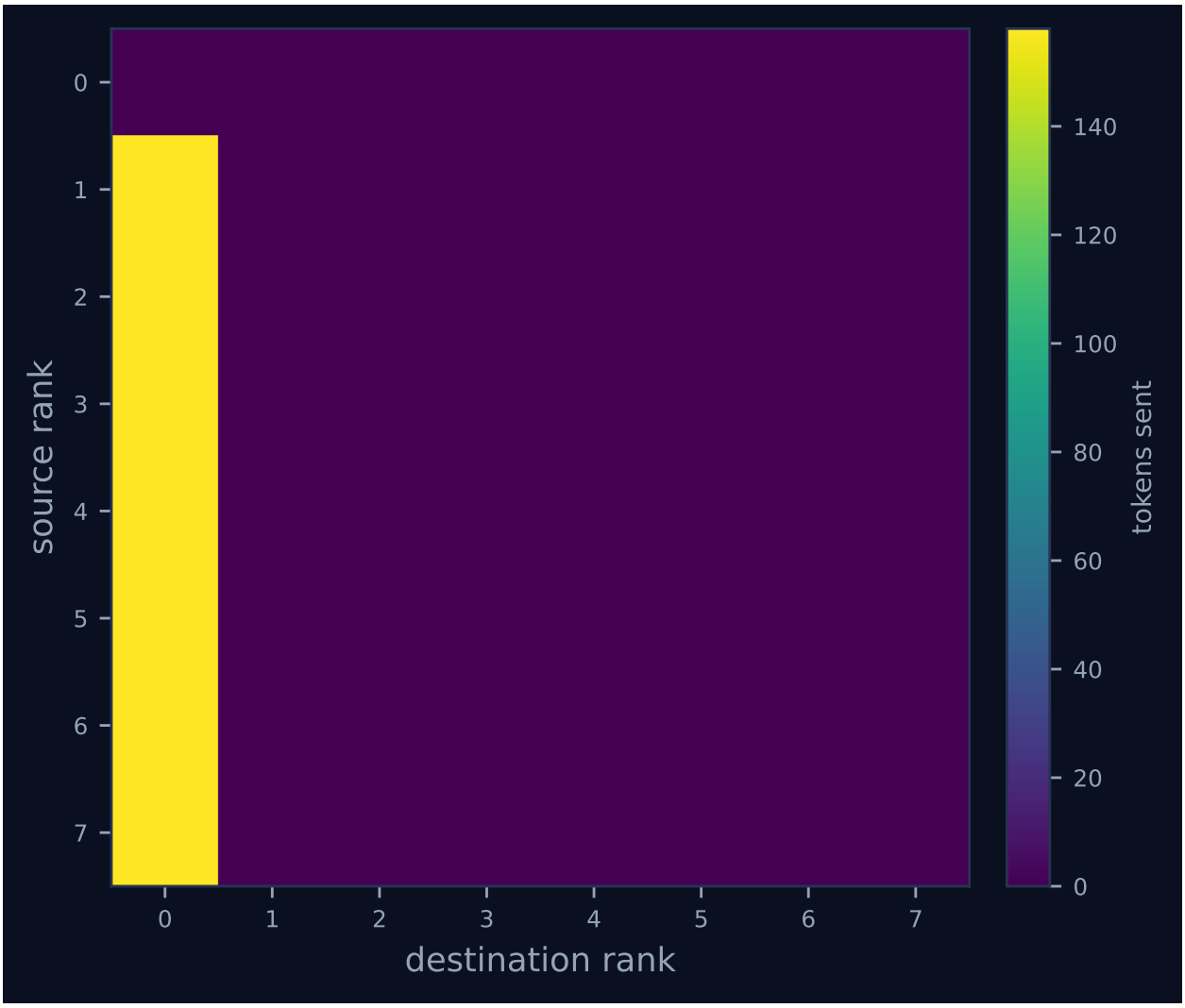

Figure 4: Communication matrix: tokens sent from each rank to each rank. The algorithm's shape is visible here — a tree concentrates traffic on a root, a ring forms a diagonal band, and an unintended all-to-all fills the plane.

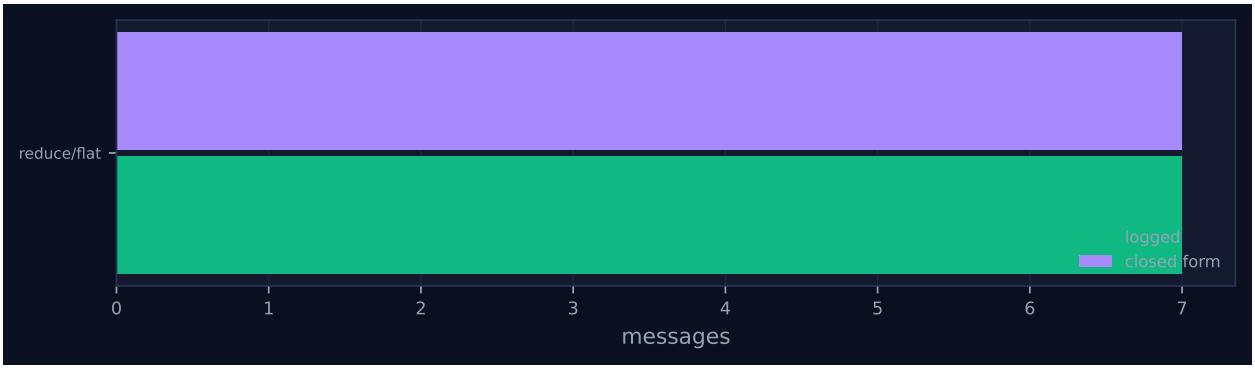

Figure 5: Logged message count against the closed-form prediction, per invocation. A red diamond marks a disagreement between the implementation and its own cost model.

6 Reading the timeline

Figure 1 is one occupied lane and seven empty ones. Rank 0 carries eight contiguous bars spanning the whole 251.64 s; ranks 1 through 7 carry two or three instantaneous ticks each inside the first 20 ms and nothing after. That is the flat reduction drawn literally: every non-root rank sends its report and returns, all seven inside 158 tokens apiece, and the root does all the work. Peak ranks busy is 1 of 8, the serial fraction of active time is 100%, load imbalance is 8.00× — the worst attainable at $p = 8$, and correctly reported — and rank 0's 209.91 s is the only non-zero entry in the per-rank table. The eighth bar is the retry: the dashboard marks it with a red failure glyph, the one item in the `failure / timeout` legend, and the stats row shows `CONTRACT VIOLATIONS 1`.

The bar widths are what to read, and they say the opposite of what a growing-prompt story would predict. The eight calls took 16.51, 20.95, 55.54, 27.65, 28.06, 20.68, 21.14 and 19.37 s on prompts of 446, 574, 712, 735, 722, 722, 722 and 722 tokens. After the third merge the prompt stops growing — it is pinned at 722 tokens, being a 437-token accumulator plus a 158-token leaf plus the template — because the accumulator has hit the ceiling and cannot get any larger. A flat fold whose prompt stops growing is a flat fold that has stopped accumulating. The four terminal calls are also the fastest and the most uniform of the run, 19–28 s, which is what an operator does when it copies its left input out again. Of the 251.6 s, 209.9 s is generation and 39.2 s is waiting for a worker to claim a queued call, which I obtained by differencing each `broker.enqueue` → `broker.claim` pair in the log; the run is therefore not distorted by pool starvation the way its `sat-mb` chain cousin is.

The communication matrix, Figure 4, is a single column of seven edges at exactly 158 tokens each, 1,106 in total. That is the least fabric traffic of the four cells — against the tree's 1,806, the k-ary tree's 1,537 and the chain's 2,448 — because only raw leaf reports ever travel and no accumulator is ever sent anywhere. The trade is exactly the one the cost model predicts: one communication round bought at the price of putting all seven applications on one rank's critical path, giving the second-worst wall time in the campaign and the highest prompt-token total, 4,643.

7 Interpretation

The configuration is by construction and the checks pass: seven messages against a closed form of $p - 1 = 7$, one round against 1, `model_checks_agree` 1 of 1. The construction most worth naming is the one the hypothesis usually gets wrong about this algorithm. A flat reduce is often described as folding all p inputs at the root in a single step, which would make it the shallowest possible arrangement and the natural upper bound on retention. The implementation does no such thing: the `flat` branch of `reduce` gathers all p contributions, sorts them by source rank because the operator is declared non-commutative, and then applies the *binary* kernel $p - 1$ times in sequence, setting `fold_depth = p - 1 = 7` and `rounds = 1`. The trace confirms it: seven distinct labels `merge:d1` through `merge:d7` on rank 0. This campaign therefore contains three distinct depths, 2, 3 and 7, not four, and this cell shares its depth with the chain.

The enforcement regime is the second thing to establish, and the fabric settles it unambiguously. Every `agent_calls` row records `max_tokens: 450` on the contract together with a non-empty `semantics` string instructing the operator to "omit whole items rather than abbreviating or re-encoding any". That is the enforced form, and this is the run where the enforcement path fires. Attempt 1 of `merge:d3` returned 472 tokens carrying 40 identifiers over ranks 0–3; the runtime rejected it, appended the diagnosis to the prompt, and attempt 2 returned 437 tokens carrying 37. The three items that enforcement removed were the difference between a 5% overrun and

compliance. The contrast with the advisory regime is stark and is measured elsewhere in my set: the `sat-mb-fidelity` flat cell ran the same p, the same twelve items per rank and the same nominal 450-token budget with `max_tokens: null`, emitted 698 tokens at the root — 55% over — and scored retention 1.000. A budget that is not checked is not a budget, and until it is checked the experiment cannot test anything.

What emerged is the erasure, and this cell is where the mechanism is unarguable because the fold advances one rank at a time. Reconstructing every intermediate from the store: `merge:d1` gives 24 identifiers over ranks $\{0, 1\}$ at 289 tokens; `merge:d2` gives 36 over $\{0, 1, 2\}$ at 426; `merge:d3` attempt 2 gives 37 over $\{0, 1, 2, 3\}$ at 437; and `merge:d4`, `d5`, `d6` and `d7` each give 37 over $\{0, 1, 2, 3\}$ at 437, the same digest each time. The cause is a division. An incompressible item is `[F-0-0] serial 597623 checksum LVPG`, nine tokens of content with no shorter equivalent, 11.81 tokens once quoted into the JSON array, so a 450-token ceiling admits $450/11.81 = 38.1$ items and the accumulator saturates at 37. From that point the reduction has stopped reducing and is merely forwarding, and because the operator writes items in the order it receives them and the runtime sorts contributions by source rank, what survives is a strict prefix: ranks 0–2 complete, plus `F-3-0`, and nothing else. That is what `rank_position_correlation` $= -0.863$ is measuring. The identical 37 survivors appear in the binomial, chain and k-ary siblings at depths 3, 7 and 2 — I compared the sets directly and the symmetric difference is empty — so depth costs nothing under a uniform enforced budget and collective selection is a pure cost decision. These four runs are the sole source of `tab_erasure` and of the macros `\NErasedRanks` and `\NErasureCorr`.

Of the flagged findings, the serial-execution warning is expected rather than a defect — a flat reduce puts every application at the root by definition — and it is worth keeping precisely because a reader meeting this pattern in a real harness should be told. The $8.00\times$ imbalance is honest and is this algorithm's true cost. The contract violation and the second attempt are the enforcement mechanism working, not a fault. The calibration warning is real and should be believed: with eight calls spanning only 446 to 735 prompt tokens the least-squares fit produced a negative intercept, the guard rejected it, and any latency prediction from this run's α and β is a median rather than a model. What is not flagged and should be is the run's central result: four of eight operator applications changed nothing, and no counter anywhere in the system notices that a lossy reduction has become a no-op. An operator that returns its left operand unchanged is exactly the signal a harness could cheaply detect — compare the result digest against the accumulator digest — and it is not detected.

8 Threats to this reading

The executor is real (`broker` $\times$ 8, `worker` $\times$ 31), so this is model behaviour rather than a surrogate implementing a loss model, and the shared-pool threat applies. This run's terminal blob is shared with the binomial and k-ary cells, and the four cells ran sequentially against one pool of Cursor subagents. I ran the determinism test: prompt digest `baffd868ca96` appears in both this run's `merge:d1` and the binomial cell's, and the two results differ (289 against 288 tokens, different digests), so the executor is not deterministic and determinism cannot explain the identities. What can is that the surviving items are verbatim high-entropy strings in rank order, so once the cut lands at 37 there is one canonical string to return. That reading also explains why the identity is confined to the saturated accumulators and does not extend to the unsaturated intermediates. I cannot exclude pool memory, but it has nothing left to account for.

The strongest internal threat is that the four terminal no-ops could be an artefact of how `flat`

presents its operands rather than a property of saturation. The kernel renders the accumulator as `REPORT A` and the incoming leaf as `REPORT B`, so a model that simply echoed `REPORT A` would produce this trace whether or not it was budget-bound. Three observations argue against that. The first three merges did integrate their right operand, growing $24 \to 36 \to 37$ identifiers with the same kernel and the same rendering. The no-ops begin exactly when the accumulator first touches 437 tokens and not before. And the same 37-item cut appears in the binomial and k-ary cells, where the operands are two subtree accumulators of comparable size rather than a large accumulator and a small leaf. Saturation explains all three; operand-position bias explains only the third-to-last.

Third, one trial, and the token constant is computed with `tokenizer_exact: false`, using the repository's approximate counter — the same one the contract check uses, which is what makes the arithmetic self-consistent internally even if it would not transfer to a real tokeniser. The $0.0574 is modelled from the calibration block, not billed. Finally, `fact_retention` counts an identifier wherever it appears in the result text, and the merge prompt's instruction line contains the literal `[F-3-2]`, which is inside the benchmark's own identifier space. I checked this run's result: rank 3's survivor is `F-3-0`, so no phantom is inflating the score here. It is a latent hazard in the metric rather than a fault in this measurement, but it does inflate the surrogate campaigns.